PYTHON ACTIVITY-BASED LEARNING

Complete Case Study Report Collection

Name: I.Rajeev

Roll Number: 26001462

Class / Section: CSD-2

10 Selected Case Studies

No.	Case Study No.	Topic
1	12	Attendance Recovery Computation
2	13	ATM Transaction Sequence Simulation
3	17	Student Performance Trend Analysis
4	27	Advanced Numeric Pattern Generator
5	29	Password Policy Validation Engine
6	35	Text Compression Using Character Runs
7	42	Student Marks Matrix Analysis
8	49	Tuple-Based Product Catalogue Audit
9	53	Food Waste Monitoring System
10	63	Multi-Product Price Fluctuation Analysis

Each report includes: problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, sample output, and conclusion.

Case Study 12: Attendance Recovery Computation

1. Problem Statement

Calculate the attendance percentage of a student and determine how many additional classes the student must attend consecutively to reach a target attendance percentage, assuming no further absences.

2. Objectives

• Calculate current attendance percentage.

• Find the minimum additional classes needed to reach the target percentage.

• Display the result clearly.

3. Assumptions

• Total classes conducted is greater than zero.

• Classes attended is between zero and total classes conducted.

• Target attendance is greater than the current percentage and at most 100% for recovery calculation.

4. Algorithm

• Start.

• Input classes conducted, classes attended, and target percentage.

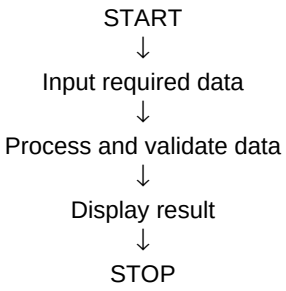
• Compute current percentage = attended ÷ conducted × 100.

• If current percentage is already at or above target, report zero additional classes.

• Otherwise find the smallest whole number x such that (attended + x) ÷ (conducted + x) × 100 ≥ target.

• Display the current percentage and x; stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT conducted, attended, target
  current ← attended / conducted * 100
  IF current >= target THEN
      extra ← 0
  ELSE
      extra ← CEILING((target * conducted / 100 - attended) /
                      (1 - target / 100))
  ENDIF
  OUTPUT current, extra
END
```

7. Python Source Code

```
import math

conducted = int(input("Classes conducted: "))
attended = int(input("Classes attended: "))
target = float(input("Target attendance (%): "))

if conducted <= 0 or attended < 0 or attended > conducted:
    print("Invalid attendance data")
```

```
elif not (0 < target <= 100):
    print("Target must be greater than 0 and at most 100")
else:
    current = attended / conducted * 100
    if current >= target:
        extra = 0
    elif target == 100:
        extra = "not reachable unless all previous classes were attended"
    else:
        extra = math.ceil((target * conducted / 100 - attended) /
                          (1 - target / 100))
    print(f"Current attendance: {current:.2f}%")
    print("Additional classes needed:", extra)
```

8. Test Cases

Test input	Expected result
100	70 | 75 | 70.00% | 20
80	64 | 75 | 80.00% | 0
50	20 | 60 | 40.00% | 25

9. Sample Output

```
Classes conducted: 100
Classes attended: 70
Target attendance (%): 75
Current attendance: 70.00%
Additional classes needed: 20
```

10. Conclusion

The program computes current attendance and estimates the minimum consecutive attended classes required to reach the target.

Case Study 13: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a basic ATM session by processing a sequence of balance enquiries, deposits, and withdrawals while preventing withdrawals that exceed the available balance.

2. Objectives

• Process a sequence of transactions.

• Update and display the balance after each transaction.

• Reject invalid transactions and insufficient-fund withdrawals.

3. Assumptions

• The opening balance is non-negative.

• Deposits and withdrawals must be positive amounts.

• Only balance enquiry, deposit, and withdrawal are supported.

4. Algorithm

• Start.

• Input opening balance and number of transactions.

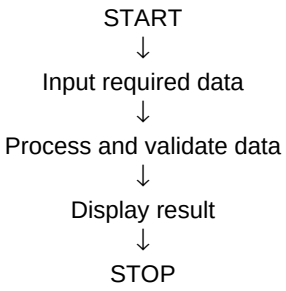
• For each transaction, input its type and amount when needed.

• For a deposit, add the amount; for a withdrawal, subtract only if funds are sufficient.

• For balance enquiry, display the current balance.

• Display the final balance and stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT balance, count
  REPEAT count times
    INPUT transaction_type
    IF type = "D" THEN INPUT amount; balance ← balance + amount
    ELSE IF type = "W" THEN
      INPUT amount
      IF amount <= balance THEN balance ← balance - amount
      ELSE OUTPUT "Insufficient funds"
      ENDIF
    ELSE IF type = "B" THEN OUTPUT balance
    ELSE OUTPUT "Invalid transaction"
    ENDIF
  END REPEAT
  OUTPUT balance
END
```

7. Python Source Code

```
balance = float(input("Opening balance: ■"))
count = int(input("Number of transactions: "))

if balance < 0 or count < 0:
```

```
    print("Invalid opening balance or transaction count")
else:
    for i in range(count):
        kind = input(f"Transaction {i+1} (D=deposit, W=withdraw, B=balance): ").upper()
        if kind == "B":
            print(f"Current balance: ■{balance:.2f}")
        elif kind in ("D", "W"):
            amount = float(input("Amount: ■"))
            if amount <= 0:
                print("Amount must be positive")
            elif kind == "D":
                balance += amount
                print(f"Deposited. Balance: ■{balance:.2f}")
            elif amount > balance:
                print("Insufficient funds")
            else:
                balance -= amount
                print(f"Withdrawn. Balance: ■{balance:.2f}")
        else:
            print("Invalid transaction type")
    print(f"Final balance: ■{balance:.2f}")
```

8. Test Cases

Test input	Expected result
1000	D 200 | 1200
1000	W 300 | 700
100	W 150 | 100 (withdrawal rejected)

9. Sample Output

```
Opening balance: ■1000
Number of transactions: 2
Transaction 1 (D=deposit, W=withdraw, B=balance): D
Amount: ■200
Deposited. Balance: ■1200.00
Transaction 2 (D=deposit, W=withdraw, B=balance): W
Amount: ■300
Withdrawn. Balance: ■900.00
Final balance: ■900.00
```

10. Conclusion

The simulation demonstrates transaction sequencing and basic balance validation. It is an educational model, not a real banking system.

Case Study 17: Student Performance Trend Analysis

1. Problem Statement

Analyse a student's marks across multiple assessments and identify the trend in performance.

2. Objectives

• Calculate average marks.

• Compare the first and last assessment scores.

• Report whether performance improved, declined, or remained unchanged.

3. Assumptions

• At least one assessment mark is entered.

• Marks are between 0 and 100.

4. Algorithm

• Start.

• Input the number of assessments and each mark.

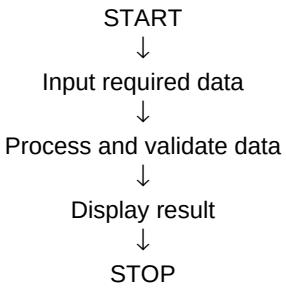
• Validate that marks are between 0 and 100.

• Calculate the average.

• Compare the first and last marks to identify the trend.

• Display average and trend; stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT n
  INPUT marks[0 ... n-1]
  average ← SUM(marks) / n
  IF marks[last] > marks[first] THEN trend ← "Improved"
  ELSE IF marks[last] < marks[first] THEN trend ← "Declined"
  ELSE trend ← "Unchanged"
  OUTPUT average, trend
END
```

7. Python Source Code

```
n = int(input("Number of assessments: "))

if n <= 0:
    print("Enter at least one assessment")
else:
    marks = []
    for i in range(n):
        mark = float(input(f"Assessment {i+1} mark (0-100): "))
        if not 0 <= mark <= 100:
            print("Invalid mark; enter a value from 0 to 100")
            break
        marks.append(mark)
    if len(marks) == n:
```

```
average = sum(marks) / n
if marks[-1] > marks[0]:
    trend = "Improved"
elif marks[-1] < marks[0]:
    trend = "Declined"
else:
    trend = "Unchanged"
print(f"Average marks: {average:.2f}")
print("Performance trend:", trend)
```

8. Test Cases

Test input	Expected result
60,70,80	70.00 | Improved
85,75,65	75.00 | Declined
70,70,70	70.00 | Unchanged

9. Sample Output

```
Number of assessments: 3
Assessment 1 mark (0-100): 60
Assessment 2 mark (0-100): 70
Assessment 3 mark (0-100): 80
Average marks: 70.00
Performance trend: Improved
```

10. Conclusion

The program summarises assessment results and classifies the trend by comparing the first and last marks.

Case Study 27: Advanced Numeric Pattern Generator

1. Problem Statement

Generate a numeric pattern with a user-selected number of rows, where each row displays an increasing sequence of numbers.

2. Objectives

• Accept a positive row count.

• Generate a structured number pattern using loops.

• Display the pattern neatly.

3. Assumptions

• The row count is a positive integer.

• The example pattern starts at 1 on each row and ends at that row number.

4. Algorithm

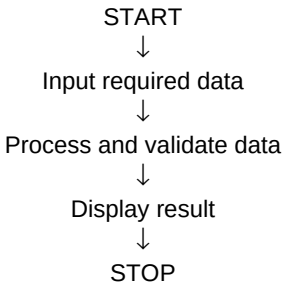
• Start.

• Input the number of rows.

• For each row from 1 to n, print the numbers from 1 to the row number.

• Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT n
  FOR row ← 1 TO n
    FOR col ← 1 TO row
      OUTPUT col without newline
    END FOR
    OUTPUT newline
  END FOR
END
```

7. Python Source Code

```
n = int(input("Number of rows: "))

if n <= 0:
    print("Enter a positive number of rows")
else:
    for row in range(1, n + 1):
        for col in range(1, row + 1):
            print(col, end=" ")
        print()
```

8. Test Cases

Test input	Expected result
3	1 / 1 2 / 1 2 3

Test input	Expected result
1	1
5	five rows ending in 1 2 3 4 5

9. Sample Output

```
Number of rows: 4
1
1 2
1 2 3
1 2 3 4
```

10. Conclusion

Nested loops generate a predictable numeric pattern and demonstrate iteration.

Case Study 29: Password Policy Validation Engine

1. Problem Statement

Check whether a password meets a basic policy requiring a minimum length, an uppercase letter, a lowercase letter, a digit, and a special character.

2. Objectives

• Check password length.

• Check for required character categories.

• Display each failed rule and the final result.

3. Assumptions

• The policy requires at least 8 characters.

• A special character is any non-alphanumeric character.

4. Algorithm

• Start.

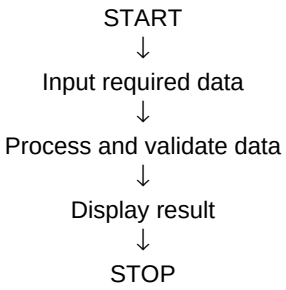
• Input a password.

• Check length, uppercase, lowercase, digit, and special character conditions.

• Display the status of each condition.

• Report valid only if all conditions pass; stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT password
  valid ← TRUE
  CHECK length >= 8
  CHECK contains uppercase, lowercase, digit, special character
  IF every check passes THEN OUTPUT "Password meets policy"
  ELSE OUTPUT "Password does not meet policy"
  ENDIF
END
```

7. Python Source Code

```
password = input("Enter password: ")

checks = {
    "At least 8 characters": len(password) >= 8,
    "Uppercase letter": any(c.isupper() for c in password),
    "Lowercase letter": any(c.islower() for c in password),
    "Digit": any(c.isdigit() for c in password),
    "Special character": any(not c.isalnum() for c in password),
}

for rule, passed in checks.items():
    print(f"{rule}: {'Pass' if passed else 'Fail'}")

if all(checks.values()):
```

```
    print("Password meets policy")
else:
    print("Password does not meet policy")
```

8. Test Cases

Test input	Expected result
Study@123	All pass | Valid
hello	Several fail | Invalid
ABCDEFG1!	Lowercase fails | Invalid

9. Sample Output

```
Enter password: Study@123
At least 8 characters: Pass
Uppercase letter: Pass
Lowercase letter: Pass
Digit: Pass
Special character: Pass
Password meets policy
```

10. Conclusion

The validator checks common password composition rules. A real system should also protect passwords securely and avoid storing them in plain text.

Case Study 35: Text Compression Using Character Runs

1. Problem Statement

Compress a string by replacing each consecutive run of the same character with the character followed by the number of repetitions.

2. Objectives

• Identify consecutive repeated characters.

• Encode each run as a character and count.

• Display original and compressed strings.

3. Assumptions

• The input is a text string.

• The simple run-length encoding format is used; characters are case-sensitive.

4. Algorithm

• Start.

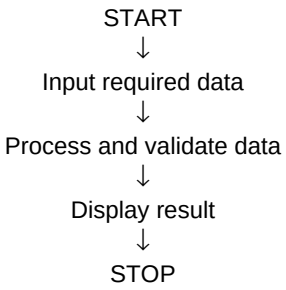
• Input a string.

• Scan the string from left to right, counting each run of identical characters.

• Append each character and its run count to the result.

• Display the compressed string; stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT text
  compressed ← ""
  i ← 0
  WHILE i < length(text)
    count ← 1
    WHILE i + count < length(text) AND text[i] = text[i+count]
      count ← count + 1
    END WHILE
    compressed ← compressed + text[i] + count
    i ← i + count
  END WHILE
  OUTPUT compressed
END
```

7. Python Source Code

```
text = input("Enter text: ")
compressed = ""
i = 0

while i < len(text):
    count = 1
    while i + count < len(text) and text[i] == text[i + count]:
        count += 1
    compressed += text[i] + str(count)
```

```
    i += count

print("Original text:", text)
print("Compressed text:", compressed)
```

8. Test Cases

Test input	Expected result
aaabbc	a3b2c1
abcd	a1b1c1d1
AAAA	A4

9. Sample Output

```
Enter text: aaabbc
Original text: aaabbc
Compressed text: a3b2c1
```

10. Conclusion

Run-length encoding is simple and effective for repeated characters, although it can make short strings longer.

Case Study 42: Student Marks Matrix Analysis

1. Problem Statement

Store marks for multiple students across multiple subjects in a matrix and calculate each student's total and average.

2. Objectives

• Represent marks as a two-dimensional list.

• Calculate row totals and averages.

• Display a summary for each student.

3. Assumptions

• At least one student and one subject are entered.

• Each mark is between 0 and 100.

4. Algorithm

• Start.

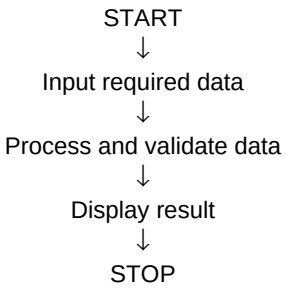
• Input number of students and subjects.

• Input marks row by row.

• For each student, calculate total and average.

• Display the summary; stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT students, subjects
  FOR each student
    INPUT marks for every subject
    total ← SUM(student marks)
    average ← total / subjects
    OUTPUT total, average
  END FOR
END
```

7. Python Source Code

```
students = int(input("Number of students: "))
subjects = int(input("Number of subjects: "))

if students <= 0 or subjects <= 0:
    print("Counts must be positive")
else:
    matrix = []
    valid = True
    for i in range(students):
        row = []
        for j in range(subjects):
            mark = float(input(f"Student {i+1}, subject {j+1} mark: "))
            if not 0 <= mark <= 100:
                print("Marks must be from 0 to 100")
                valid = False
```

```
            break
        row.append(mark)
    if not valid:
        break
    matrix.append(row)

if valid:
    for i, row in enumerate(matrix, start=1):
        total = sum(row)
        average = total / subjects
        print(f"Student {i}: Total={total:.2f}, Average={average:.2f}")
```

8. Test Cases

Test input	Expected result
80,70,90	Total 240 | Average 80
50,50,50	Total 150 | Average 50
100,90	Total 190 | Average 95

9. Sample Output

```
Number of students: 2
Number of subjects: 3
Student 1, subject 1 mark: 80
Student 1, subject 2 mark: 70
Student 1, subject 3 mark: 90
Student 2, subject 1 mark: 60
Student 2, subject 2 mark: 75
Student 2, subject 3 mark: 85
Student 1: Total=240.00, Average=80.00
Student 2: Total=220.00, Average=73.33
```

10. Conclusion

A two-dimensional list organises marks efficiently and makes per-student totals and averages easy to calculate.

Case Study 49: Tuple-Based Product Catalogue Audit

1. Problem Statement

Audit a product catalogue stored as tuples by checking product IDs for duplicates and identifying products with invalid prices.

2. Objectives

• Store product details as tuples.

• Identify duplicate product IDs.

• Flag zero or negative prices.

3. Assumptions

• Each product tuple contains an ID, name, and price.

• A valid price must be greater than zero.

4. Algorithm

• Start.

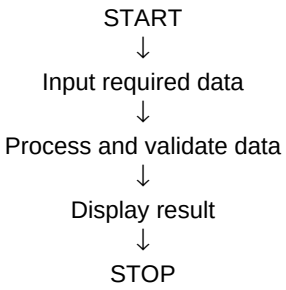
• Input the number of products and their ID, name, and price.

• Store each product as a tuple.

• Count IDs and check each price.

• Display duplicate IDs and invalid-price products; stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT n
  products ← empty list
  REPEAT n times
    INPUT id, name, price
    APPEND (id, name, price) to products
  END REPEAT
  FIND IDs occurring more than once
  FIND products where price <= 0
  OUTPUT audit findings
END
```

7. Python Source Code

```
products = []
n = int(input("Number of products: "))

for i in range(n):
    product_id = input("Product ID: ").strip()
    name = input("Product name: ").strip()
    price = float(input("Price: ■"))
    products.append((product_id, name, price))

ids = [p[0] for p in products]
duplicates = sorted({pid for pid in ids if ids.count(pid) > 1})
invalid_prices = [p for p in products if p[2] <= 0]
```

```
print("Catalogue:")
for product in products:
    print(product)
print("Duplicate product IDs:", duplicates if duplicates else "None")
print("Products with invalid prices:",
      invalid_prices if invalid_prices else "None")
```

8. Test Cases

Test input	Expected result
P1, Pen, 10; P1, Pencil, 0	Duplicate ID P1 | Pencil invalid price
A1, Book, 50	No duplicates | No invalid prices
X, Bag, -5	No duplicates | Bag invalid price

9. Sample Output

```
Number of products: 2
Product ID: P1
Product name: Pen
Price: ■10
Product ID: P1
Product name: Pencil
Price: ■0
Catalogue:
('P1', 'Pen', 10.0)
('P1', 'Pencil', 0.0)
Duplicate product IDs: ['P1']
Products with invalid prices: [('P1', 'Pencil', 0.0)]
```

10. Conclusion

The audit uses tuples to preserve product records and identifies duplicate identifiers and invalid prices.

Case Study 53: Food Waste Monitoring System

1. Problem Statement

Calculate leftover meals and the percentage of food wasted by comparing the number of meals prepared with the number served.

2. Objectives

• Find the number of leftover meals.

• Calculate food waste as a percentage.

• Display a clear summary.

3. Assumptions

• Meals prepared is greater than zero.

• Meals served is between zero and meals prepared.

4. Algorithm

• Start.

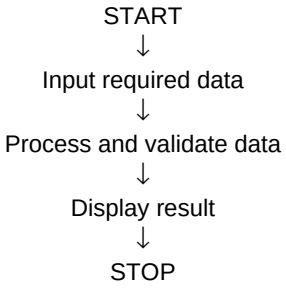
• Input meals prepared and meals served.

• Calculate leftover = prepared – served.

• Calculate waste percentage = leftover ÷ prepared × 100.

• Display leftover and waste percentage; stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT prepared, served
  leftover ← prepared - served
  waste ← (leftover / prepared) * 100
  OUTPUT leftover, waste
END
```

7. Python Source Code

```
prepared = int(input("Meals prepared: "))
served = int(input("Meals served: "))

if prepared <= 0 or served < 0 or served > prepared:
    print("Invalid meal counts")
else:
    leftover = prepared - served
    waste = leftover / prepared * 100
    print("Leftover meals:", leftover)
    print("Waste percentage:", round(waste, 2), "%")
```

8. Test Cases

Test input	Expected result
100	80 | 20 | 20.0%

Test input	Expected result
50	50 | 0 | 0.0%
200	150 | 50 | 25.0%

9. Sample Output

```
Meals prepared: 100
Meals served: 80
Leftover meals: 20
Waste percentage: 20.0 %
```

10. Conclusion

The program calculates leftover meals and food-waste percentage using basic arithmetic operations. It can help identify days when food waste is high.

Case Study 63: Multi-Product Price Fluctuation Analysis

1. Problem Statement

Track prices of multiple products across two observations and calculate the change and percentage change for each product.

2. Objectives

• Record original and new prices for several products.

• Calculate absolute and percentage price changes.

• Classify each product as increased, decreased, or unchanged.

3. Assumptions

• Prices are non-negative and original prices are greater than zero for percentage calculations.

• Each product has an original price and a new price.

4. Algorithm

• Start.

• Input the number of products.

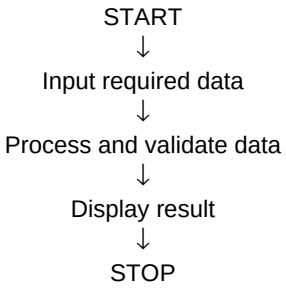
• For each product, input its name, original price, and new price.

• Calculate change = new price – original price.

• Calculate percentage change = change ÷ original price × 100.

• Classify and display the result; stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT n
  REPEAT n times
    INPUT name, old_price, new_price
    change ← new_price - old_price
    percent ← change / old_price * 100
    IF change > 0 THEN status ← "Increased"
    ELSE IF change < 0 THEN status ← "Decreased"
    ELSE status ← "Unchanged"
    OUTPUT name, change, percent, status
  END REPEAT
END
```

7. Python Source Code

```
n = int(input("Number of products: "))

if n <= 0:
    print("Enter at least one product")
else:
    for i in range(n):
        name = input("Product name: ")
        old_price = float(input("Original price: ■"))
        new_price = float(input("New price: ■"))
```

```
if old_price <= 0 or new_price < 0:
    print("Prices must be non-negative and original price > 0")
    continue

change = new_price - old_price
percent = change / old_price * 100
if change > 0:
    status = "Increased"
elif change < 0:
    status = "Decreased"
else:
    status = "Unchanged"

print(f"{name}: Change ■{change:.2f} ({percent:.2f}%), {status}")
```

8. Test Cases

Test input	Expected result
Rice, 50, 55	■5 | 10.00% Increased
Milk, 60, 54	-■6 | -10.00% Decreased
Bread, 30, 30	■0 | 0.00% Unchanged

9. Sample Output

```
Number of products: 1
Product name: Rice
Original price: ■50
New price: ■55
Rice: Change ■5.00 (10.00%), Increased
```

10. Conclusion

The program compares product prices and calculates both absolute and percentage changes to summarise price movement.